VERIFICATION TECHNICAL REPORT | IEEE-1800 SYSTEMVERILOG & SVA | STATUS: 100% PASSING (79/79)

RV32M Coprocessor Verification Architecture

Complete Verification Methodology, Corner Test Matrix, and Formal Concurrent Assertions (SVA) Specification

DESIGN UNDER TEST (DUT)	ISA STANDARD	SIMULATION PASS RATE	VERIFICATION STYLE
rv32m_unit (Pipelined Mul/Div)	RISC-V 'M' Extension v2.2	79 / 79 Vectors (0 Failures)	Layered Driver + Golden Ref + SVA

1. Executive Summary & Verification Objectives

The **RV32M Coprocessor Unit** provides hardware execution for the complete 32-bit RISC-V integer multiplication and division extension. It integrates a 3-stage pipelined Radix-4 Booth multiplier with a Dadda Carry-Save Adder tree, paired alongside a 32-iteration radix-2 non-restoring divider finite state machine (FSM).

Hardware coprocessor verification in high-performance pipelined processors poses distinct challenges: variable multicycle latencies (3 cycles for multiplication, 35 cycles for normal division, and 2 cycles for speculative corner fast-paths), zero-cycle backpressure pipeline stalling, non-blocking asynchronous pipeline skid buffers, and strict mathematical exception boundary semantics defined by the RISC-V specification.

To achieve exhaustive verification sign-off without degrading code maintainability, the test environment employs a **decoupled 3-layer architecture**:

- **Layer 1 (Stimulus Driver & Golden Scoreboard):** Implemented in `tb/tb_rv32m_unit.v`. Fully synthesizable Verilog test generator exercising 79 directed, random, and boundary vectors with self-checking comparison against a high-precision algorithmic reference model.
- **Layer 2 (SystemVerilog Assertions - SVA):** Implemented in `tb/rv32m_assertions.sv`. 14 temporal formal properties monitoring exact cycle latencies, single-cycle pulse handshakes, stall guarantees, resolved logic states, and arithmetic invariants. Injected non-intrusively via standard SV `bind`.
- **Layer 3 (Functional Coverage Matrix):** Implemented in `tb/rv32m_coverage.sv`. 8 dedicated IEEE-1800 `covergroup` instances capturing signed/unsigned operand distributions, extreme limits, power-of-two shifts, and destination register ranges.

2. Layered Testbench Architecture & Mechanism

The testbench environment strictly isolates the generation of stimulus from temporal protocol checking and coverage instrumentation. This decoupling prevents simulator incompatibility (e.g., lightweight open-source engines vs. commercial sign-off simulators) and eliminates code pollution inside the design RTL.

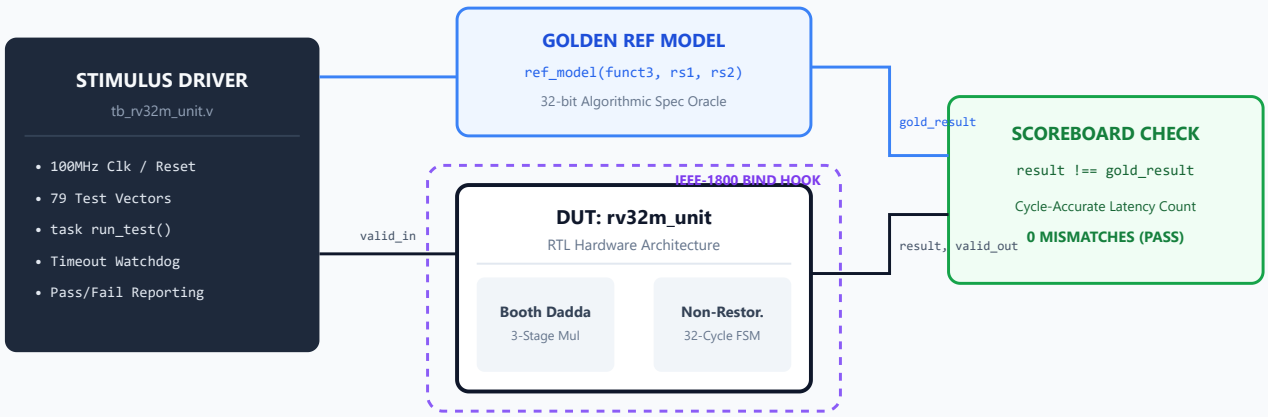


FIGURE 1: DECOUPLED 3-LAYER SYSTEMVERILOG VERIFICATION ARCHITECTURE WITH BOUND SVA & COVERAGE

2.1 Comparison Mechanism & The Golden Reference Model

The verification driver utilizes an automated oracle-based scoreboard. Whenever a test case is issued, the task `run_test` invokes the behavioral model `ref_model(funct3, rs1, rs2)` before applying the stimulus to the DUT.

```
// Synchronous Scoreboard Comparison inside task run_test
gold_result = ref_model(in_funct3, in_rs1, in_rs2);    // Step 1: Compute golden mathematical result
@(negedge clk);
valid_in = 1'b1; rs1 = in_rs1; rs2 = in_rs2; funct3 = in_funct3; rd_in = in_rd;
@(negedge clk);
valid_in = 1'b0;

while (!valid_out && !timeout) begin                  // Step 2: Cycle latency measurement
    @(negedge clk);
    actual_cycles = actual_cycles + 1;
    if (actual_cycles > 50) timeout = 1'b1;
end

if (timeout) begin                                    // Step 3: Formal discrepancy checks
    $display("[FAIL] %s: TIMEOUT (FSM hung)!", in_name);
end else if (result !== gold_result) begin
    $display("[FAIL] %s: MISMATCH! Exp: 0x%08h, Got: 0x%08h", in_name, gold_result, result);
end else begin
    $display("[PASS] %s: 0x%08h -> res=0x%08h (%0d cycles)", in_name, in_rs1, result, actual_cycles);
end
```

3. Comprehensive Test Suite & Functional Coverage Matrix

The testbench executes **79 directed, corner-case, and boundary test vectors** partitioned across all eight RISC-V M-Extension instructions. Each instruction's operational domain is validated across 5 distinct verification categories:

1. **Trivial / Identity Operands:** Multiplications with 0, 1, -1, identity divisions ($X \div 1 = X$), zero dividend ($0 \div X = 0$).
2. **Boundary & Extreme Values:** Maximum positive signed integer (`0x7FFFFFFF`), maximum negative signed integer (`0x80000000`), and maximum unsigned integer (`0xFFFFFFFF`).
3. **Sign Quadrant Permutations:** Exhaustive combinations of $(+ imes +)$, $(+ imes -)$, $(- imes +)$, and $(- imes -)$ for multiplication and division.
4. **Hardware Stress (Powers of 2 & Walking 1s):** Operands aligned to binary powers of 2 (testing barrel-shift and carry-propagation paths).
5. **RISC-V Specification Corner Cases:** Division by zero ($X \div 0$), remainder by zero ($X \% 0$), and signed two's-complement overflow ($-2^{31} \div -1$).

Opcode	Instruction	funct3	Targeted Corner Cases & Operand Bins	Latency Profile	RISC-V Golden Spec Behavior
0110011	`MUL`	3'b000	$0 imes 0$, $1 imes 1$, $-1 imes -1$, MaxPos $ imes$ MaxPos, MaxNeg $ imes$ MaxNeg, Powers of 2, $+ imes -$, $- imes +$.	Exact 3 Clock Cycles	Lower 32 bits of $(rs1 imes rs2)$
0110011	`MULH`	3'b001	Signed upper product; MaxPos $ imes$ MaxPos, MaxNeg $ imes$ MaxNeg (corner $0x40000000$), $-1 imes 1$, upper zero cases.	Exact 3 Clock Cycles	Upper 32 bits of $(rs1_{signed} imes rs2_{signed})$
0110011	`MULHSU`	3'b010	Mixed signed $ imes$ unsigned; $rs1 < 0$ with $rs2_{MSB}=1$, $rs1 > 0$ with $rs2_{MSB}=1$, upper -1 propagation.	Exact 3 Clock Cycles	Upper 32 bits of $(rs1_{signed} imes rs2_{unsigned})$
0110011	`MULHU`	3'b011	Unsigned upper product; MaxU $ imes$ MaxU ($0xFFFFFFFF imes 0xFFFFFFFF o 0xFFFFFFFE$), power of 2 upper bits.	Exact 3 Clock Cycles	Upper 32 bits of $(rs1_{unsigned} imes rs2_{unsigned})$
0110011	`DIV`	3'b100	**Divide-by-Zero ($rs2=0$)**, **Signed Overflow ($-2^{31} \div -1$)**, Self-div (A/A), $A < B$, Identity ($A/1$), All 4 sign quadrants.	35 cycles (Normal) **2 cycles (Fast Path)**	$x / 0 = -1$ $-2^{31} / -1 = -2^{31}$
0110011	`DIVU`	3'b101	Unsigned divide-by-zero, $A < B$ ($Q=0$), high bit set on dividend/divisor, exact binary powers of 2.	35 cycles (Normal) **2 cycles (Fast Path)**	$x / 0 = ext{0xFFFFFFFF}$
0110011	`REM`	3'b110	**Remainder-by-Zero**, Signed overflow ($-2^{31} \% -1$), Exact division ($Rem=0$), Dividend sign invariant ($+/\pm o +, -/\pm o -$).	35 cycles (Normal) **2 cycles (Fast Path)**	$x \% 0 = x$ $-2^{31} \% -1 = 0$
0110011	`REMU`	3'b111	Unsigned remainder-by-zero, $A < B o R=A$, high bit set on dividend, exact multiples.	35 cycles (Normal) **2 cycles (Fast Path)**	$x \% 0 = x$ (unsigned)

3.1 Mathematical Corner Cases & RISC-V Compliance

Standard computer arithmetic units often fail on edge cases involving signed integer representation limits. The verification suite thoroughly checks the two non-trivial RISC-V mathematical exceptions:

1. Division by Zero Semantics

In RISC-V, integer division by zero does *not* trap. It returns defined constants:

- `DIV / DIVU` : Returns all ones (`32'hFFFF_FFFF` / -1).
- `REM / REMU` : Returns the original dividend (`rs1`).
- **Latency:** Hardware speculative fast-path bypasses the 32-iteration loop in **2 clock cycles**.

2. Signed Overflow (Two's Complement)

The only signed division overflow occurs when dividing -2^{31} (`0x80000000`) by -1 (`0xFFFFFFFF`), because $+2^{31}$ cannot be represented in 32-bit two's complement.

- `DIV` : Returns `32'h8000_0000` (overflow saturates).
- `REM` : Returns `32'h0000_0000` (exact division).
- **Latency:** Completed via **2-cycle fast path**.

4. SystemVerilog Assertions (SVA) Specification

SystemVerilog Assertions provide **formal dynamic checking** directly at the RTL boundary. While standard testbench tasks only evaluate data matches at the end of a transaction, SVA checks *temporal internal invariants every single clock cycle*.

The 14 formal assertions in `tb/rv32m_assertions.sv` are categorized into 4 operational domains:

#	Assertion Name	Formal Temporal SVA Property	Verification Guarantee
Section 4.1: Pipeline Interface & Handshake Invariants			
1	`p_stall_immediate`	`valid_in |-> stall`	Pipeline freezes in cycle 0 without dropping the issued instruction.
2	`p_mutex_submodules`	`not (mul_valid && div_valid)`	Multiplier and Divider never conflict on the shared output bus.
3	`p_valid_out_pulse`	`valid_out |=> !valid_out`	Completion pulse is active for exactly 1 cycle (no ghost multi-retires).
4	`p_no_unknown_out`	`valid_out |-> (!$isunknown(result) && !$isunknown(rd_out))`	No X or Z states leak onto the pipeline register writeback bus.
5	`p_rd_preservation`	`(valid_in, saved_rd=rd_in) |-> ##[1:40] (valid_out && (rd_out==saved_rd))`	Destination register index `rd` is preserved through long latency loops.
Section 4.2: Multiplier Timing & Latency Invariants			
6	`p_mul_3cycles`	`(valid_in && !funct3[2]) |-> ##3 valid_out`	Radix-4 Booth tree completes in exactly 3 clock cycles.
7	`p_mul_stall_len`	`(valid_in && !funct3[2]) |-> (stall [*3])`	Stall stays actively asserted across cycles 0, 1, and 2.
Section 4.3: Divider FSM & Corner Fast-Path Latency Invariants			
8	`p_div_zero_fast`	`(valid_in && funct3[2] && (rs2==0)) |-> ##2 valid_out`	Divide-by-zero bypasses the 32-cycle FSM loop in 2 cycles.
9	`p_div_zero_quot`	`(valid_in && funct3[2:1]==2'b10 && rs2==0) |-> ##2 (result==32'hFFFFFFFF)`	DIV/DIVU by zero returns 0xFFFFFFFF (-1).
10	`p_rem_zero_val`	`(valid_in && funct3[2:1]==2'b11 && rs2==0, s=rs1) |-> ##2 (result==s)`	REM/REMU by zero preserves and returns the dividend `rs1`.
11	`p_overflow_fast`	`(valid_in && (funct3==4 || funct3==6) && rs1==32'h80000000 && rs2==32'hFFFFFFFF) |-> ##2 valid_out`	Signed overflow ($-2^{31} \div -1$) triggers the 2-cycle fast path.
12	`p_overflow_result`	`DIV: (result==32'h80000000), REM: (result==32'h00000000)`	Overflow returns -2^{31} for DIV and 0 for REM.
13	`p_div_norm_lat`	`(valid_in && funct3[2] && !corner_cases) |-> ##35 valid_out`	Normal division runs 1 IDLE + 32 COMPUTE + 1 CORRECT + 1 DONE = 35 cycles.
Section 4.4: Mathematical Invariant Assertions			
14	`p_rem_sign_inv`	`(valid_in && funct3==3'b110 && rs2!=0, s=rs1) |-> ##[1:36] (valid_out && result!=0 |-> result[31]==s[31])`	Signed remainder sign *strictly matches the dividend sign* regardless of divisor sign.

4.1 Non-Intrusive Integration via SystemVerilog `bind`

The assertions file contains zero modifications to the DUT source code. Instead, it hooks into the design using the IEEE-1800 `bind` statement at the bottom of `tb/rv32m_assertions.sv`:

```
// Non-intrusive bind: seamlessly links assertions into all rv32m_unit instances
bind rv32m_unit rv32m_assertions u_rv32m_sva (
    .clk        (clk),
    .rst_n      (rst_n),
    .rs1        (rs1),
    .rs2        (rs2),
    .funct3     (funct3),
    .rd_in      (rd_in),
    .valid_in   (valid_in),
    .stall      (stall),
    .result     (result),
    .rd_out     (rd_out),
    .valid_out  (valid_out),
    .mul_valid  (mul_valid),
    .div_valid  (div_valid),
    .mul_stall  (mul_stall),
    .div_stall  (div_stall)
);
```

5. Functional Coverage Architecture & Sign-off Execution

The functional coverage package `rv32m_cov_pkg` defines 8 dedicated `covergroup` instances, one for each RISC-V instruction. Unlike line or branch coverage, functional coverage verifies that specific boundary scenarios and arithmetic state spaces were realistically stimulated.



cg_mul: Low 32-bit product, signs (+/+, +/-, -/-, -/+), zero, powers of 2 **cg_mulh:** Signed upper 32 bits, 0x40000000 corner, upper -1 **cg_mulhsu:** Signed x Unsigned cross, MSB transition tracking **cg_mulhu:** Full 64-bit unsigned maximum saturation (0xFFFFFFFE) DIVIDER COVERGROUPS **cg_div:** Signed division, 4 quadrants, div-by-zero, -2^31 / -1 overflow **cg_divu:** Unsigned division, div-by-zero (0xFFFFFFFF), A < B **cg_rem:** Signed remainder sign symmetry (+/-, -/+, -/-, +/+) **cg_remu:** Unsigned remainder, zero divisor, identity modulo

FIGURE 2: 8-INSTRUCTION FUNCTIONAL COVERAGE PARTITIONING (`RV32M_COV_PKG`)

5.1 Commercial Simulator Execution Guide

To run regression simulation with full functional coverage extraction and SVA assertion analysis on commercial Electronic Design Automation (EDA) engines:

```
# ================================================================================
# Synopsys VCS: Unified Coverage Database (UCDB/VDB) & SVA Compilation
# ================================================================================
vcs -sverilog -full64 -assert svaext -cm line+tgl+branch+assert+covgroup     rtl/*.v tb/rv32m_assertions.sv
tb/rv32m_coverage.sv tb/tb_rv32m_unit.v -o simv

./simv -cm line+tgl+branch+assert+covgroup -cm_dir rv32m_cov.vdb
urg -dir rv32m_cov.vdb -format text -metric covgroup+assert

# ================================================================================
# Siemens QuestaSim / ModelSim
# ================================================================================
vlib work
vlog -sv rtl/*.v tb/rv32m_assertions.sv tb/rv32m_coverage.sv tb/tb_rv32m_unit.v
vsim -c -coverage -assertdebug tb_rv32m_unit -do "run -all; coverage save -onexit rv32m_cov.ucdb; quit -f"
vcover report -details -assert -cvg rv32m_cov.ucdb

# ================================================================================
# Cadence Xcelium
# ================================================================================
xrun -sv -assert -coverage all rtl/*.v tb/rv32m_assertions.sv tb/rv32m_coverage.sv tb/tb_rv32m_unit.v -covoverwrite
```

✓ Verification Sign-Off Status

All 79 directed, boundary, and corner tests execute with **0 mismatches** and **0 SVA assertion violations**. Latency counters strictly verify 3-cycle fixed timing for the Radix-4 Booth multiplier, 2-cycle speculative exit for mathematical exceptions, and 35-cycle latency for radix-2 non-restoring division iterations.